

Your screenshot checklist

Verified Docker Desktop run | 2026-09-23 | UTC and Pakistan Standard Time (UTC+05:00)

SCREENSHOT CHECKLIST - CAMPUS ROOM BOOKING

All 15 screenshots have been supplied and inserted. For any replacement capture, save the PNG in:

C:\Users\Hamza\Desktop\ESD\Assignment_1\screenshots

Keep the exact filenames below.

Keep Docker Desktop and Codex open while capturing. Open the link for each screenshot below; the prepared links choose the correct historical time range. Wait until the charts or results appear. Use Windows + Shift + S, select the required area, open the capture notification, press Ctrl+S, and save with the EXACT filename shown. If a save dialog adds .png automatically, do not type .png twice.

Grafana login, if asked: admin / local-coursework-change-me (the documented local demonstration account). For Kibana, choose Campus request logs if prompted and dismiss any notice covering the results. Grafana displays UTC; Kibana may display Pakistan local time. Pakistan Standard Time is UTC+05:00; both ranges are listed below. Capture data soon, before the configured retention removes it. Do not replace historical ranges with Last 15 minutes.

The services/test/concurrency pages are labelled snapshots of actual command results, not live terminals or fabricated images. Grafana, Prometheus and Kibana links open the real applications. If you zoom out to fit a dashboard, press Ctrl+0 when finished.

Screenshot 1: 01-docker-services.png

Open: <http://localhost:8011/go/01>

Page/panel/query: Recorded Docker service verification

Time: Use the fresh verification time shown on the page.

Do: Open the link and capture the Services evidence card; it is a clearly labelled real command snapshot.

Must show: All eight required service names, healthy status, the actual docker compose ps command and capture time.

Why: Proves the complete assignment stack was running.

Screenshot 2: 02-api-swagger.png

Open: <http://localhost:8011/go/02>

Page/panel/query: Swagger: GET /rooms

Time: Current time; no historical filter.

Do: Expand GET /rooms, click Try it out, then Execute; capture the 200 response and rooms.

Must show: API title, GET /rooms, HTTP 200 and the returned rooms.

Why: Shows the application is usable and backed by real data.

Screenshot 3: 03-prometheus-targets.png

Open: <http://localhost:8011/go/03>

Page/panel/query: Prometheus Targets

Time: Current time.

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: campus-app, node-exporter and prometheus UP; cardinality-demo should also be UP.

Why: Proves the metric collectors are connected.

Screenshot 4: 04-grafana-overview.png

Open: <http://localhost:8011/go/04>

Page/panel/query: Campus application: panels 1-6

Time: UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464

Do: Open the prepared range; capture the top six panels. Close the side menu if it takes up space.

Must show: Requests per second, successful requests, 5xx/error percentage and p95/p99 charts with actual traces.

Why: Shows the core observability dashboard.

Screenshot 5: 05-grafana-throughput.png

Open: <http://localhost:8011/go/05>

Page/panel/query: Campus load: Server throughput

Time: UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: Server throughput title, request-rate trace, req/s units and selected UTC time range.

Why: Records real throughput under the fresh k6 baseline.

Screenshot 6: 06-grafana-p95-p99.png

Open: <http://localhost:8011/go/06>

Page/panel/query: Campus load: Server p95 / p99

Time: UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: Both p95 and p99 traces, latency units, legend and selected UTC range.

Why: Shows tail latency rather than relying only on averages.

Screenshot 7: 07-grafana-business-metrics.png

Open: <http://localhost:8011/go/07>

Page/panel/query: Campus application: booking panels 11-16

Time: UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464

Do: Scroll to Booking creation rate; capture panels 11-16 together. Use Ctrl+- once or twice if needed, then Ctrl+0 afterwards.

Must show: Booking creation, cancellation, conflicts and Booking conflict percentage (own exploration).

Why: Demonstrates domain-specific metrics and the derived exploration metric.

Screenshot 8: 08-node-exporter.png

Open: <http://localhost:8011/go/08>

Page/panel/query: Campus node: all six panels

Time: UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464

Do: Capture all six panels; use Ctrl+- if needed. This is the Docker Linux/WSL2 environment, not native Windows monitoring.

Must show: CPU, memory, disk usage, disk I/O, network receive/transmit and Linux VM label.

Why: Proves resource data exists and identifies the Linux/WSL2 scope.

Screenshot 9: 09-kibana-request-id.png

Open: <http://localhost:8011/go/09>

Page/panel/query: Kibana Discover: Campus request logs

Time: UTC: 2026-09-23 04:19:25.674 to 2026-09-23 04:20:25.674; Pakistan time: 2026-09-23 09:19:25.674 to 2026-09-23 09:20:25.674

Do: Open the prepared search. If prompted, choose Campus request logs. Expand the matching row with the arrow on its left.

Must show: Request ID verify-0c74ea91-41be-4bab-864b-80388a2d07d4; method, route, status_code, duration_ms, severity and message. Expand the matching document.

Why: Proves a fresh request travelled through Filebeat into searchable structured logs.

Screenshot 10: 10-anomaly-baseline-fault-recovery.png

Open: <http://localhost:8011/go/10>

Page/panel/query: Campus load: Server p95 / p99 across all three phases

Time: UTC: 2026-09-23 04:22:08.939 to 2026-09-23 04:38:46.082; Pakistan time: 2026-09-23 09:22:08.939 to 2026-09-23 09:38:46.082

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: Low baseline latency, increased fault latency and lower recovery latency, with the UTC axis and legend.

Why: Shows the reproduced problem and recovery in one chart. Quiet gaps are intentional cooldowns.

Screenshot 11: 11-cardinality-100-series.png

Open: <http://localhost:8011/go/11>

Page/panel/query: Prometheus Table; query: count(demo_requests_total{job="cardinality-demo"})

Time: UTC: 2026-09-23 04:39:46.493 to 2026-09-23 04:39:46.493; Pakistan time: 2026-09-23 09:39:46.493 to 2026-09-23 09:39:46.493

Do: Open the prepared historical query; if the result is not shown, click Execute. Do not switch the time to Now.

Must show: The complete query, historical evaluation time and result 100.

Why: Compares unique-ID label growth with the correct bounded metric.

Screenshot 12: 12-cardinality-1-series.png

Open: <http://localhost:8011/go/12>

Page/panel/query: Prometheus Table; query: count(demo_requests_total{job="cardinality-demo"})

Time: UTC: 2026-09-23 04:39:55.419 to 2026-09-23 04:39:55.419; Pakistan time: 2026-09-23 09:39:55.419 to 2026-09-23 09:39:55.419

Do: Open the prepared historical query; if the result is not shown, click Execute. Do not switch the time to Now.

Must show: The complete query, historical evaluation time and result 1.

Why: Compares unique-ID label growth with the correct bounded metric.

Screenshot 13: 13-tests-passed.png

Open: <http://localhost:8011/go/13>

Page/panel/query: Recorded PostgreSQL test output

Time: Fresh test run shown on the page.

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: Actual pytest progress and 26 passed in 3.46s; the Docker test command and evidence location.

Why: Shows the complete suite passed against real PostgreSQL.

Screenshot 14: 14-concurrency-test.png

Open: <http://localhost:8011/go/14>

Page/panel/query: Recorded k6 concurrent-booking result

Time: Fresh concurrency run shown on the page.

Do: Open the link. Wait for the page to finish loading, then capture the indicated content.

Must show: 1 winner, 19 conflicts, passed checks and the command used.

Why: Proves competing requests cannot double-book the same slot.

Screenshot 15: 15-fault-request-id.png

Open: <http://localhost:8011/go/15>

Page/panel/query: Kibana Discover: matching injected-delay and completion records

Time: UTC: 2026-09-23 04:29:25.313 to 2026-09-23 04:31:28.704; Pakistan time: 2026-09-23 09:29:25.313 to 2026-09-23 09:31:28.704

Do: Expand the fault warning and completion records with the same request ID; show delay_ms and duration_ms.

Must show: Request ID b5d0153e-b4a9-425b-a30a-73942337346d, fault_delay_injected with delay_ms=500 and the matching slow request_completed duration.

Why: Connects the latency increase to the injected fault, rather than guessing its cause.

Place replacement screenshots in the screenshots folder using the specified filenames, then tell me: 'Insert my screenshots and regenerate the final PDFs.'